第13章 数字签名

0. 学习目标总览

教材给出本章七个学习目标：

1. 概述数字签名过程。
2. 理解 ElGamal 数字签名方案。
3. 理解 Schnorr 数字签名方案。
4. 理解 NIST 数字签名方案。
5. 比较 NIST 数字签名方案、ElGamal 数字签名方案和 Schnorr 数字签名方案的异同。
6. 理解椭圆曲线数字签名方案。
7. 理解 RSA-PSS 数字签名方案。

数字签名利用签名者的私钥生成签名，利用相应公钥验证签名。它能够认证消息来源和消息完整性，并为发生争执时的第三方仲裁提供依据。本章介绍的方案都遵循 **“先对消息求哈希，再使用私钥相关算法生成短签名”** 的基本模型。

```
数字签名
  |
  ├── 一般过程
  |     ├── 签名：M → H(M) → 私钥签名算法 → S
  |     `-- 验证：M → H(M)；结合 S 和公钥判断有效性
  |
  ├── 基于离散对数
  |     ├── ElGamal
  |     ├── Schnorr
  |     `-- DSA
  |
  ├── 基于椭圆曲线离散对数
  |     `-- ECDSA
  |
  `-- 基于 RSA
        `-- RSA-PSS：哈希 + 盐 + MGF + RSA 运算
```

1. 学习目标 1：数字签名过程

1.1 一般模型

假设 Bob 要向 Alice 发送消息 M，消息不要求保密，但 Alice 需要确认消息确由 Bob 发出且未被修改。基本过程如下：

1. Bob 计算消息哈希值 $h = H(M)$。
2. Bob 将 h 和自己的私钥输入数字签名生成算法，得到签名 S。
3. Bob 发送 $M \parallel S$。
4. Alice 对收到的消息重新计算 $H(M)$。

5. Alice 将消息哈希值、签名 S 和 Bob 的公钥输入验证算法。
6. 验证算法返回签名有效或无效。

只有 Bob 掌握其私钥，其他人不能生成能够通过 Bob 公钥验证的签名；若消息被修改，重新计算的哈希值也会改变，原签名不能通过验证。因此，数字签名同时认证发送源和数据完整性。

图 13.1 所示为构建和使用数字签名过程的一般模型。本章介绍的所有数字签名方案都以这个基本结构为基础。

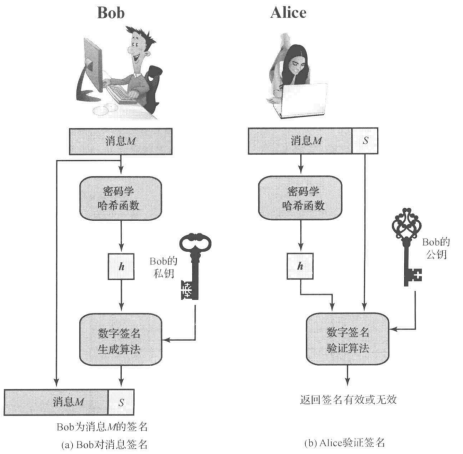


图 13.1　构建和使用数字签名过程的一般模型

图 13.1 构建和使用数字签名过程的一般模型

1.2 数字签名的性质

使用共享密钥的消息认证可以抵抗第三方攻击，却不能充分解决通信双方彼此不信任的问题：接收方也拥有共享密钥，因此能够伪造消息；发送方也可能否认曾经发送过消息。

数字签名必须具有下列性质：

1. 能够验证签名者以及签名的日期和时间。
2. 能够认证被签名消息的内容。
3. 出现争执时，签名能够由第三方仲裁。

数字签名由签名者独有的私钥生成，而验证所需的公钥可以公开，因此验证者能够检查签名，却不能据此生成签名。

1.3 攻击类型与伪造结果

教材按攻击者获得的能力列出五类攻击：

攻击类型	攻击者掌握的条件
唯密钥攻击	只知道被攻击用户的公钥。
已知消息攻击	掌握若干消息及其有效签名。
一般选择消息攻击	在知道具体用户公钥之前选定消息，再取得这些消息的有效签名。

攻击类型	攻击者掌握的条件
定向选择消息攻击	在知道目标用户公钥后选定消息，再取得签名。
适应性选择消息攻击	把签名者作为“先知”，后续签名请求可以依赖先前取得的消息-签名对。

攻击成功的结果按危害程度可分为：

结果	含义
完全破译	确定签名者的私钥。
通用伪造	获得等效签名算法，可为任意消息构造签名。
选择伪造	能够为攻击者选定的特定消息伪造签名。
存在性伪造	至少能伪造一条消息的签名，但不能控制该消息。

1.4 数字签名的要求

一个数字签名方案应满足：

1. 签名必须依赖被签名消息的位模式。
2. 签名必须使用只有发送方知道的信息，以防止伪造和否认。
3. 数字签名应容易生成。
4. 数字签名应容易识别和验证。
5. 无论是由给定签名伪造消息，还是由给定消息伪造签名，都应在计算上不可行。
6. 数字签名应便于保存副本。

安全哈希函数是满足这些要求的重要基础，但签名方案的具体结构仍必须经过仔细设计。

1.5 直接数字签名

直接数字签名只涉及发送方和接收方，并假设接收方已知发送方的公钥。若还要求保密性，可以使用共享对称密钥加密消息和签名，但应当遵循：

```
消息 M
  → 先生成内部签名 S
  → 再对 M || S 进行外层加密
```

签名必须先于外层加密。发生争执时，接收方可以保存并向第三方提供明文消息及其签名；若先加密再对密文签名，第三方还必须知道解密密钥才能读取原消息。

直接签名的有效性依赖于发送方私钥的安全。发送方可能声称私钥早已丢失或被盗，攻击者也可能在盗取私钥后为伪造消息加入更早的时间戳。因此，需要通过私钥管理、消息时间戳、密钥泄露报告、数字证书和证书授权等机制降低风险。

2. 学习目标 2：ElGamal 数字签名方案

ElGamal 数字签名建立在有限域离散对数问题上。设 q 为素数，α 是 q 的本原根。对任意整数 i, j，有

$$\alpha^{i} \equiv \alpha^{j} \pmod{q} \quad \Longleftrightarrow \quad i \equiv j \pmod{q-1}$$

这一性质是签名验证正确性的基础。

2.1 密钥生成

用户 A 按以下步骤生成密钥：

1. 随机选择私钥 X_A，满足 $1 < X_A < q-1$。
2. 计算

$$Y_A = \alpha^{X_A} \bmod q$$

3. 私钥为 X_A，公钥为 $\{q, \alpha, Y_A\}$。

由 X_A 计算 Y_A 容易，而由公开的 Y_A 求 X_A 是离散对数问题。

2.2 签名生成

首先计算消息哈希 $m = H(M)$，并把 m 视为 $0 \le m \le q-1$ 的整数。然后：

1. 选择随机整数 K，满足

$$1 \le K \le q-1, \qquad \gcd(K, q-1) = 1$$

2. 计算

$$S_1 = \alpha^{K} \bmod q$$

3. 计算 $K^{-1} \bmod (q-1)$。
4. 计算

$$S_2 = K^{-1}(m - X_A S_1) \bmod (q-1)$$

5. 输出签名 (S_1, S_2)。

2.3 签名验证

验证者计算

$$V_1 = \alpha^{m} \bmod q$$

$$V_2 = Y_A^{S_1} S_1^{S_2} \bmod q$$

当且仅当 $V_1 = V_2$ 时接受签名。因为

$$Y_A^{S_1} S_1^{S_2} = \alpha^{X_A S_1} \alpha^{K S_2} = \alpha^{X_A S_1 + K S_2} \pmod{q}$$

而签名公式给出

$$K S_2 \equiv m - X_A S_1 \pmod{q-1}$$

所以指数满足 $X_A S_1 + K S_2 \equiv m \pmod{q-1}$，最终有 $V_2 = \alpha^{m} \bmod q = V_1$。

2.4 教材示例

取 $q=19$、本原根 $\alpha=10$。Alice 选择 $X_A=16$，则

$$Y_A=10^{16}\bmod 19=4$$

对哈希值 $m=14$ 的消息，选择 $K=5$：

$$S_1=10^5\bmod 19=3,\qquad K^{-1}\bmod 18=11$$

$$S_2=11(14-16\times 3)\bmod 18=4$$

验证时：

$$V_1=10^{14}\bmod 19=16$$

$$V_2=4^3\times 3^4\bmod 19=16$$

两者相等，因此签名 $(3,4)$ 有效。

3. 学习目标 3：Schnorr 数字签名方案

Schnorr 方案同样基于离散对数问题。它的主要特点是签名计算量较少，而且主要指数运算不依赖消息，可以在处理器空闲时预先完成。

3.1 全局参数与密钥生成

1. 选择素数 p 和 q，使 q 是 $p-1$ 的素因子，即 $q\mid(p-1)$。
2. 选择整数 a，使

   $$a^q\equiv 1\pmod{p}$$

   全局参数为 (p,q,a)。
3. 随机选择私钥 s，满足 $0<s<q$。
4. 计算公钥

   $$v=a^{-s}\bmod p$$

教材给出的典型长度为 $p\approx 2^{1024}$、$q\approx 2^{160}$。

3.2 签名生成

1. 选择随机整数 r，$0<r<q$，并预先计算

   $$x=a^r\bmod p$$

2. 将 x 与消息连接，计算挑战值

   $$e=H(M\parallel x)$$

3. 计算响应值

   $$y=(r+se)\bmod q$$

4. 输出签名 (e,y)。

3.3 签名验证

验证者先重构

$$x' = a^y v^e \bmod p$$

再检验

$$e \overset{?}{=} H(M \parallel x')$$

正确性来自

$$x' equiv a^y v^e \equiv a^y a^{-se} \equiv a^{y-se} \equiv a^r \equiv x \pmod{p}$$

所以合法签名必然得到 $H(M \parallel x') = H(M \parallel x) = e$。

4. 学习目标 4：NIST 数字签名方案 DSA

数字签名算法（DSA）是 NIST 发布的联邦信息处理标准 FIPS 186 中的签名方案。DSA 使用安全哈希算法，并建立在离散对数计算的困难性以及 ElGamal、Schnorr 方法的基础上。

DSA 只提供数字签名功能。它虽然属于公钥技术，但不能用于加密或密钥交换。

成有效的签名。

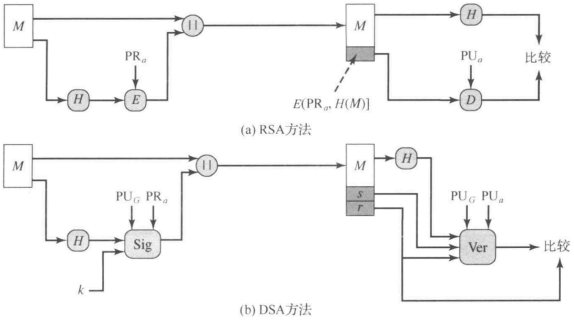


图 13.2　两种数字签名方法

图 13.2 RSA 方法与 DSA 方法的签名结构比较

4.1 全局参数与用户密钥

一组用户共享全局公开参数 (p, q, g)：

1. q 是一个 N 位素数。
2. p 是一个素数，且 $q \mid (p-1)$。
3. 选择适当的 h，计算

$$g = h^{(p-1)/q} \bmod p, \qquad g > 1$$

每个用户随机或伪随机选择私钥 x，满足 $0 < x < q$，并计算公钥

$$y = g^x \bmod p$$

由 x 计算 y 容易，而由 y 恢复 x 等价于求模 p 的离散对数。

4.2 签名生成

对每次签名，选择唯一的随机或伪随机整数 k，满足 $0 < k < q$。签名分量为

$$r = (g^k \bmod p) \bmod q$$

$$s = \left[k^{-1}\left(H(M) + xr\right)\right] \bmod q$$

签名为 (r, s)。其中，计算量较大的 $g^k \bmod p$ 和 $k^{-1} \bmod q$ 都不依赖消息，可以预先计算。

4.3 签名验证

设接收方收到消息 M' 和签名 (r', s')，计算

$$w = (s')^{-1} \bmod q$$

$$u_1 = H(M')w \bmod q, \qquad u_2 = r'w \bmod q$$

$$v = [(g^{u_1} y^{u_2}) \bmod p] \bmod q$$

若 $v = r'$，则签名有效。

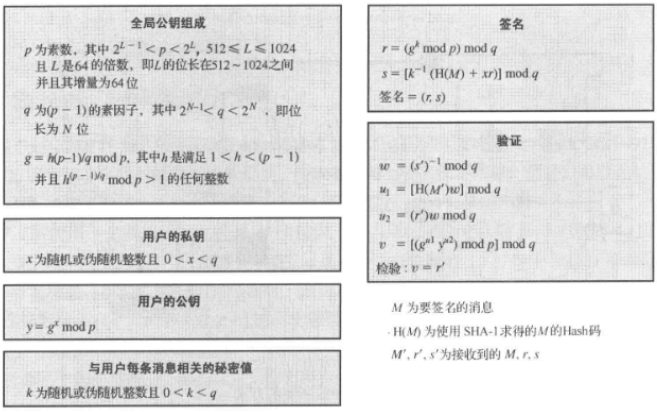


图 13.3　数字签名算法（DSA）

图 13.3 数字签名算法（DSA）

验证能够恢复 r 的原因是，将 $y = g^x$、$u_1 = H(M)w$、$u_2 = rw$ 和 $w = s^{-1}$ 代入后，指数中出现

$$u_1 + xu_2 = w(H(M) + xr) \equiv k \pmod{q}$$

于是验证端计算的值等于 $(g^k \bmod p) \bmod q = r$。由于离散对数困难，攻击者由 r 恢复 k 或由 s 恢复 x 在计算上不可行。

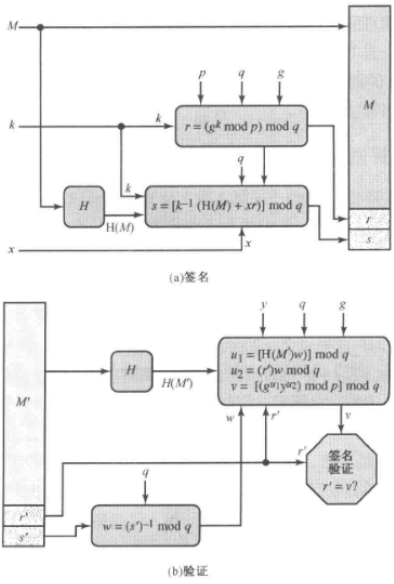


图 13.4　DSA 的签名和验证函数

图 13.4 DSA 的签名和验证函数

5. 学习目标 5：ElGamal、Schnorr 与 DSA 的比较

三种方案都以离散对数困难性为安全基础，均先对消息求哈希，并在每次签名时引入随机或伪随机整数。主要区别如下。

比较项	ElGamal 签名	Schnorr 签名	DSA
全局结构	素数 q 与本原根 α	素数 p, q，其中 $q \mid (p-1)$，以及阶为 q 的元素 a	素数 p, q，其中 $q \mid (p-1)$，以及 $g = h^{(p-1)/q} \bmod p$
私钥/公钥	X_A；$Y_A = \alpha^{X_A} \bmod q$	s；$v = a^{-s} \bmod p$	x；$y = g^x \bmod p$
每次签名随机量	K，且与 $q-1$ 互素	r	唯一的 k
签名形式	(S_1, S_2)	(e, y)	(r, s)
消息进入签名的方式	$m = H(M)$ 出现在 S_2 中	$e = H(M \parallel a^r)$	$H(M)$ 出现在 s 中
验证思想	比较 α^m 与 $Y_A^{S_1} S_1^{S_2}$	重构 $x' = a^y v^e$，再重算哈希	由 (M, r, s) 和公钥恢复候选值 v，比较 v 与 r
可预处理部分	α^K 与 K^{-1} 不依赖消息	$a^r \bmod p$ 不依赖消息，预处理优势最突出	$g^k \bmod p$ 与 k^{-1} 不依赖消息

Schnorr 的结构把预计算值 x 纳入哈希挑战 e；DSA 则让 r 只依赖全局参数和随机量 k，再利用 s 把消息哈希与私钥联系起来。DSA 的验证结果最终恢复并比较签名分量 r，而不是直接比较两

个消息哈希值。

6. 学习目标 6：椭圆曲线数字签名算法 ECDSA

ECDSA 是 FIPS 186 中基于椭圆曲线密码的数字签名技术。椭圆曲线密码在较短密钥长度下具有较高效率，因此 ECDSA 得到广泛采用。

6.1 全局域参数与密钥生成

ECDSA 使用素数域 Z_q 上的椭圆曲线，全局参数为：

- q：素数。
- a, b：定义曲线

  $$y^2 = x^3 + ax + b$$

  的 Z_q 上整数。
- $G = (x_g, y_g)$：曲线上的基点。
- n：点 G 的阶，即满足 $nG = O$ 的最小正整数。

签名者选择私钥

$$d \in [1, n-1]$$

并计算公钥

$$Q = dG$$

6.2 签名生成

对消息 m：

1. 选择随机或伪随机整数 $k \in [1, n-1]$。
2. 计算点

   $$P = (x, y) = kG$$

   并令

   $$r = x \bmod n$$

   若 $r = 0$，重新选择 k。
3. 计算 $k^{-1} \bmod n$ 和消息哈希 $e = H(m)$。
4. 计算

   $$s = k^{-1}(e + dr) \bmod n$$

   若 $s = 0$，重新选择 k。
5. 输出签名 (r, s)。

6.3 签名验证

验证者拥有全局参数和签名者公钥 Q，执行：

1. 检查 r, s 是否都在 $[1, n-1]$ 内。
2. 计算 $e = H(m)$。
3. 计算

$$w = s^{-1} \bmod n$$

4. 计算

$$u_1 = ew, \qquad u_2 = rw$$

5. 计算曲线点

$$X = (x_1, y_1) = u_1 G + u_2 Q$$

6. 若 $X = O$，拒绝签名；否则令

$$v = x_1 \bmod n$$

7. 当且仅当 $v = r$ 时接受签名。

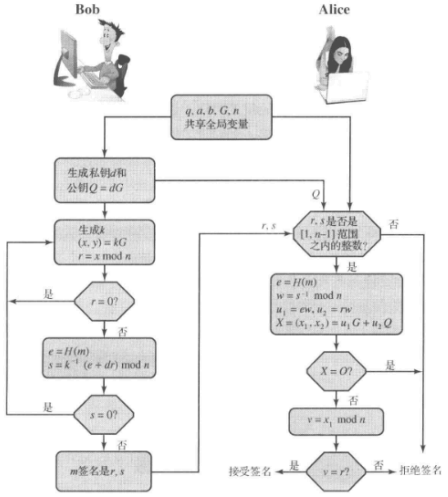


图 13.5　ECDSA 签名与验证

图 13.5 ECDSA 签名与验证

正确性可由签名公式推出：

$$s = k^{-1}(e + dr) \bmod n \quad \Longrightarrow \quad k = (s^{-1}e + s^{-1}dr) \bmod n$$

因此

$$u_1 G + u_2 Q = u_1 G + u_2 dG = (u_1 + u_2 d)G = kG$$

验证端得到的 X 与签名端的 $P = kG$ 相同，其横坐标模 n 后自然满足 $v = r$。

7. 学习目标 7：RSA-PSS 数字签名方案

RSA 概率签名方案（RSA-PSS）是基于 RSA 的随机化签名方案。不同 RSA 签名方案的主要差别，在于如何把消息哈希嵌入填充格式，以及验证时如何检查哈希值和编码消息。RSA-PSS 引入伪随机盐和掩蔽生成函数，其安全性可以与 RSA 的安全性建立联系。

7.1 掩蔽生成函数 MGF1

$\mathrm{MGF}(X, \mathrm{maskLen})$ 以任意长度字节串 X 和所需掩蔽长度 $\mathrm{maskLen}$ 为输入，输出指定长度的伪随机串。RSA-PSS 使用的 MGF1 基于安全哈希函数构造。

设哈希输出长度为 hLen 字节，令

$$k = \left\lceil \frac{\mathrm{maskLen}}{\mathrm{hLen}} \right\rceil - 1$$

对计数器 $0, 1, \ldots, k$，把计数器编码为 32 位串 C，依次连接

$$T = \mathrm{Hash}(X \,\|\, C_0) \,\|\, \mathrm{Hash}(X \,\|\, C_1) \,\|\, \cdots \,\|\, \mathrm{Hash}(X \,\|\, C_k)$$

最终取 T 的前 $\mathrm{maskLen}$ 字节作为输出。

MGF 1 的操作过程如下。若需要的输出长度等于哈希值的长度（$maskLen = hLen$），则将输入 X 的哈希值与值为 0 的 32 位计数器连接起来作为输出。若 maskLen 比 hLen 大，则 MGF 1 继续将当前处理的串 T 与计数器连接起来并对 X 计算哈希值。因此，输出为

$$\mathrm{Hash}(X \,\|\, 0) \,\|\, \mathrm{Hash}(X \,\|\, 1) \,\|\, \cdots \,\|\, \mathrm{Hash}(X \,\|\, k)$$

该过程直到 T 的长度大于等于 maskLen 时结束，此时将 T 的前 maskLen 字节作为输出。

7.2 PSS 消息编码

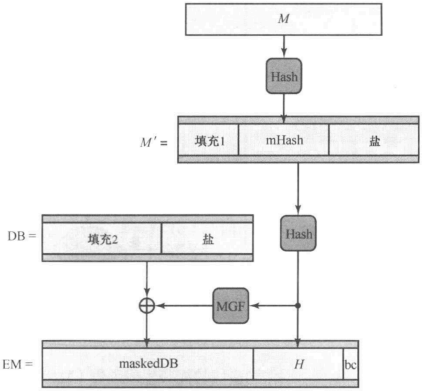


图 13.6　RSA-PSS 编码

图 13.6 RSA-PSS 编码

RSA-PSS 先把消息 M 编码为定长字节串 EM。主要参数为：

- Hash：输出长度为 hLen 字节的哈希函数。
- MGF：掩蔽生成函数，通常为 MGF1。
- sLen：伪随机盐 `salt` 的字节长度，通常等于 hLen。

- emBits：小于 RSA 模数 n 位长度的编码长度。
- $\text{emLen} = \lceil \text{emBits}/8 \rceil$：$EM$ 的字节长度。

编码过程为：

1. 计算

$$\text{mHash} = \text{Hash}(M)$$

2. 生成伪随机 `salt`，并形成

$$M' = \text{padding}_1 \parallel \text{mHash} \parallel \text{salt}$$

其中 padding_1 是 64 个 0 位。

3. 计算

$$H = \text{Hash}(M')$$

4. 形成

$$DB = \text{padding}_2 \parallel \text{salt}$$

其中 padding_2 由若干 `0x00` 和末尾的 `0x01` 组成。

5. 计算

$$\text{dbMask} = \text{MGF}(H, \text{emLen} - \text{hLen} - 1)$$

6. 计算

$$\text{maskedDB} = DB \oplus \text{dbMask}$$

7. 把 `maskedDB` 最左侧超出 emBits 的位设为 0。
8. 输出

$$EM = \text{maskedDB} \parallel H \parallel \texttt{BC}$$

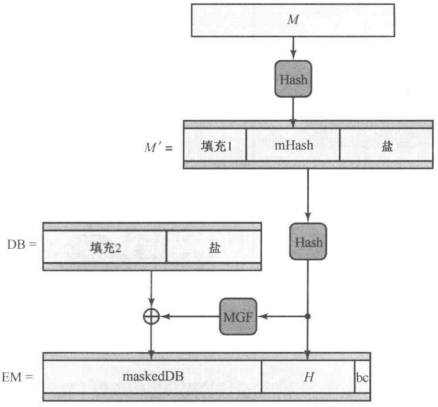


图 13.6　RSA-PSS 编码

图 13.6 RSA-PSS 编码

填充扩展增加了寻找相同摘要消息的难度。伪随机盐使同一私钥对同一消息进行两次签名时也会产生不同结果，这是 RSA-PSS 随机化处理的核心。

7.3 形成签名

签名者的 RSA 私钥为 $\{d, n\}$，公钥为 $\{e, n\}$。将 EM 解释为非负整数 m，计算

$$s = m^d \bmod n$$

再把 s 转换成与 RSA 模数相同字节长度的签名字节串 S。

7.4 签名验证

验证者先把签名 S 解释为整数 s，使用公钥恢复编码消息：

$$m = s^e \bmod n$$

再把 m 转换为长度为 emLen 的 EM。验证主线如下：

1. 检查 EM 长度是否足够，且最右侧字节是否为 `0xBC`。
2. 从 EM 中分离 `maskedDB` 和 H，并检查最左侧未使用位是否为 0。
3. 计算

   $$\mathrm{dbMask} = \mathrm{MGF}(H, \mathrm{emLen} - \mathrm{hLen} - 1)$$

   并恢复

   $$DB = \mathrm{maskedDB} \oplus \mathrm{dbMask}$$

4. 检查 DB 中的 $\mathrm{padding}_2$ 格式，从末尾取出 `salt`。
5. 对收到的消息重新计算 $\mathrm{mHash} = \mathrm{Hash}(M)$，形成

   $$M' = \mathrm{padding}_1 \,\|\, \mathrm{mHash} \,\|\, \mathrm{salt}$$

6. 计算

   $$H' = \mathrm{Hash}(M')$$

7. 当且仅当所有格式检查通过且 $H = H'$ 时，签名验证一致。

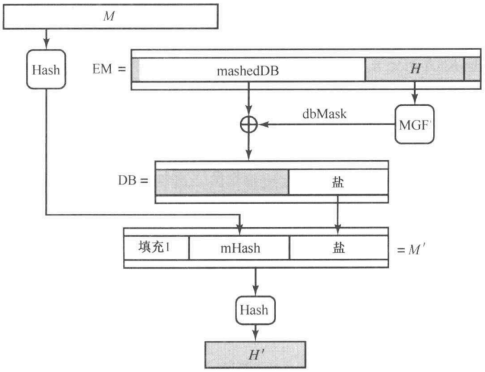


图 13.7　RSA-PSS EM 验证

图 13.7 RSA-PSS 编码消息验证

验证者知道填充常数，可以把恢复出的常数与规定格式比较，形成除 $H = H'$ 外的额外检查。验证者不需要事先知道盐值，而是从编码消息中恢复它；盐的作用与 DSA、ECDSA 中每次签名使用的随机变量相似，使同一消息的多次签名产生不同结果。

本章关键术语整理

术语	核心含义
数字签名	使用私钥生成、使用公钥验证的消息认证机制。
直接数字签名	只涉及发送方和接收方的签名方案。
唯密钥攻击	攻击者只知道目标用户公钥的攻击。
已知消息攻击	攻击者掌握若干消息及其有效签名的攻击。
选择消息攻击	攻击者可选择消息并取得相应有效签名的攻击。
存在性伪造	至少能伪造一条消息的签名，但不能控制消息内容。
ElGamal 签名	基于本原根和有限域离散对数问题的签名方案。
Schnorr 签名	通过哈希挑战 e 和响应 y 构造的离散对数签名方案。
DSA	NIST 标准化的、只用于数字签名的离散对数方案。
ECDSA	在椭圆曲线群上实现的数字签名算法。
RSA-PSS	使用随机盐、MGF 和 RSA 运算的概率签名方案。
MGF1	通过哈希函数和递增计数器生成指定长度掩蔽串的函数。
盐（salt）	RSA-PSS 编码中每次随机生成、使同一消息产生不同签名的字节串。
编码消息（EM）	RSA-PSS 中经哈希、填充、盐和掩蔽处理后送入 RSA 运算的定长串。

学习目标自检

1. 能否说明数字签名的一般生成与验证过程，以及它为何能认证消息来源和完整性？
2. 能否写出 ElGamal 的密钥生成、(S_1, S_2) 签名公式和 $V_1 = V_2$ 验证公式？
3. 能否说明 Schnorr 方案中 x、e、y 的作用，并证明验证端能够重构 x？

4. 能否写出 DSA 的全局参数、密钥、签名 (r, s) 和验证值 v 的完整计算过程？
5. 能否从参数结构、随机量、签名形式和验证方法四方面比较 ElGamal、Schnorr 与 DSA？
6. 能否写出 ECDSA 的签名与验证公式，并说明 $u_1G + u_2Q = kG$ 为什么成立？
7. 能否说明 RSA-PSS 中盐、MGF1、`maskedDB`、H 和 `0xBC` 的作用，并概括编码与验证流程？